UNIVERSITÀ POLITECNICA DELLE MARCHE

Facoltà di Ingegneria

Corso di laurea magistrale in
Ingegneria Informatica e dell'Automazione

Data Science



ChronoBot

Implementazione di un Chatbot interattivo che permette di conversare con grandi personaggi storici tramite framework Rasa

Studenti:
Diocis Daniel
Ricci Ettore
Salvatore Pietro
Sebastianelli Nicola

Anno Accademico 2025/2026

Indice

1 Introduzione **2**

2 Rasa **3**
2.1 Creazione Ambiente Virtuale 3
2.2 Altre tecnologie utilizzate 4

3 ChronoBot **5**
3.1 Il Dataset: Progettazione e Generazione della Conoscenza . . . 6
3.1.1 Generazione tramite Large Language Models (LLM) . . 6
3.1.2 Struttura della Knowledge Base 7
3.1.3 Vantaggi della Generazione Sintetica 7
3.2 Files principali 8
3.3 Sviluppo del ChatBot 9
3.3.1 Inizio e fine chat 9
3.3.2 Intent richiesta abilità chatbot 11
3.3.3 Intent di richiesta non valida 12
3.3.4 Intent di Fallback e Topic Specifici 13
3.3.5 Intent: select_character 13
3.3.6 Intent Tematici 14
3.3.7 Intent: ask_vita 14
3.3.8 Intent: ask_studio 17
3.3.9 Intent: ask_scoperte 18
3.3.10 Intent: ask_epoca 21
3.3.11 Intent: ask_curiosita 22
3.3.12 Intent: ask_citazioni 24
3.3.13 Intent: ask_relazioni 25
3.3.14 Intent: ask_eredita 27
3.3.15 Intent: ask_futuro 29
3.3.16 Intent: change_character 30
3.4 Rules e Stories 32
3.4.1 Rules 32
3.4.2 Stories 33

4 Esecuzione del chatBot **34**
4.1 Esecuzione nell'ambiente virtuale 34
4.2 Connessione a Telegram 34

1 Introduzione

Negli ultimi anni i chatbot hanno assunto un ruolo sempre più rilevante all'interno dei sistemi informatici interattivi, grazie ai significativi progressi nel campo dell'Intelligenza Artificiale e, in particolare, del Natural Language Processing (NLP). Un chatbot può essere definito come un sistema software progettato per interagire con l'utente attraverso il linguaggio naturale, simulando una conversazione umana tramite interfacce testuali o vocali. L'obiettivo principale di tali sistemi è fornire supporto, informazioni o servizi in modo automatico ed efficiente, riducendo la necessità di intervento umano diretto.

I chatbot possono essere classificati in diverse categorie in base alla complessità e alle tecniche utilizzate. I sistemi più semplici sono basati su regole predefinite e pattern di riconoscimento, mentre i chatbot più avanzati sfruttano modelli di apprendimento automatico e reti neurali profonde, come i modelli di linguaggio di grandi dimensioni, in grado di comprendere il contesto e generare risposte articolate.

Gli ambiti di utilizzo dei chatbot sono molteplici.

- In contesti aziendali e commerciali vengono impiegati per il customer support, la gestione delle richieste degli utenti e l'automazione di processi ripetitivi.
- In ambito educativo possono fungere da assistenti virtuali per l'apprendimento, fornendo spiegazioni personalizzate e supporto allo studio.
- Ulteriori applicazioni si riscontrano nel settore sanitario, nei servizi bancari, nei sistemi di prenotazione e, più in generale, in tutti quei contesti in cui è richiesta un'interazione rapida ed efficiente con l'utente.

L'adozione dei chatbot presenta numerosi **vantaggi**. Tra i principali si annoverano la disponibilità continua del servizio, la riduzione dei costi operativi e la capacità di gestire un elevato numero di richieste simultaneamente. Inoltre, l'integrazione di modelli di apprendimento automatico consente a questi sistemi di migliorare le proprie prestazioni nel tempo, adattandosi alle esigenze degli utenti e aumentando la qualità delle risposte fornite.

Tuttavia, l'utilizzo dei chatbot comporta anche alcune **criticità**. Le limitazioni nella comprensione semantica del linguaggio naturale possono portare a risposte imprecise o non adeguate al contesto. Inoltre, l'affidabilità delle informazioni fornite dipende fortemente dai dati di addestramento e dai modelli utilizzati. Altri aspetti critici riguardano la gestione della privacy, la sicurezza dei dati e il rischio di una eccessiva dipendenza da sistemi automatizzati in contesti in cui è ancora necessario il giudizio umano.

2 Rasa

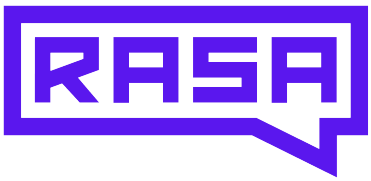


Figura 1: Logo Rasa

Rasa è un framework open-source progettato per lo sviluppo di chatbot e assistenti conversazionali intelligenti, con un forte orientamento al controllo completo del flusso conversazionale e alla gestione autonoma dei dati. Il framework si compone principalmente di due moduli:

- *Rasa NLU* si occupa della comprensione del linguaggio naturale e permette di estrarre intenti e entità dai messaggi dell'utente. Questa fase è fondamentale per interpretare correttamente le richieste e trasformare il linguaggio naturale in informazioni strutturate utilizzabili dal sistema.
- *Rasa Core* gestisce il flusso della conversazione, decidendo quale azione intraprendere in base allo stato corrente del dialogo e alle informazioni precedentemente acquisite.

Uno degli aspetti distintivi di Rasa è l'utilizzo di modelli di apprendimento automatico per la gestione del dialogo. Il comportamento del chatbot viene appreso a partire da *stories*, ovvero esempi di conversazioni che descrivono le possibili interazioni tra utente e sistema. Questo approccio consente di realizzare dialoghi flessibili e contestuali, capaci di adattarsi a differenti sequenze di input senza dover definire rigidamente ogni possibile percorso conversazionale.

Rasa supporta inoltre l'integrazione di azioni personalizzate, permettendo al chatbot di interagire con sistemi esterni, come database, API o servizi aziendali. Questa caratteristica rende il framework particolarmente adatto allo sviluppo di chatbot orientati a compiti specifici, come assistenza clienti, sistemi informativi o automazione di processi. L'architettura modulare facilita inoltre l'estensione e la manutenzione del sistema nel tempo.

2.1 Creazione Ambiente Virtuale

Per eseguire il framework *Rasa* in modo efficiente ed evitare possibili conflitti tra librerie *Python*, si è resa necessaria la creazione di un ambiente virtuale

dedicato. Un ambiente virtuale consente infatti di isolare le dipendenze del progetto dal resto del sistema. Esistono diversi strumenti per la sua creazione; in questo caso si è scelto di utilizzare il gestore di pacchetti *Anaconda* che, una volta installato, permette di generare l'ambiente virtuale desiderato tramite pochi e semplici comandi, riportati di seguito.

Scaricare e installare Anaconda, selezionando anche la spunta per aggiungerlo alle variabili di ambiente.

```
# Comandi per configurare Rasa su Windows da terminale:

#Creare ambiente virtuale chiamato Rasa_es
3.8.10 di Python
conda create --name Rasa_es python==3.8

#Attivare l'ambiente virtuale
conda activate Rasa_es

#Installare la libreria urllib3
conda install -c anaconda urllib3

#Aggiornamento versione di pip3
pip3 install -U pip3 --user

#Installazione del framework Rasa
pip3 install rasa
```

2.2 Altre tecnologie utilizzate

- **Ngrok:** Strumento che consente di esporre un server locale su una rete pubblica tramite un tunnel sicuro. È utile per testare e condividere applicazioni web ospitate localmente, permettendo l'accesso esterno tramite un URL temporaneo.

- **Telegram:** App di messaggistica con la quale si è deciso di far comunciare il chatbot sviluppato.

3 ChronoBot



Figura 2: Logo ChronoBot

Il progetto **ChronoBot** nasce dall'esigenza di creare uno strumento educativo e interattivo che consenta agli utenti di "dialogare" con alcuni dei più grandi personaggi della storia. L'obiettivo principale è rendere l'apprendimento storico più coinvolgente e accessibile attraverso l'uso dell'Intelligenza Artificiale conversazionale, permettendo di esplorare non solo le imprese pubbliche, ma anche aneddoti, relazioni personali, contesto culturale e visioni del futuro delle figure storiche più influenti dell'umanità. In questo modo, la storia viene presentata non come un insieme statico di date ed eventi, ma come un racconto vivo e dinamico.
Al momento sono disponibili 7 personaggi storici:

- **Giulio Cesare**: politico, generale e scrittore romano, figura chiave della trasformazione della Repubblica Romana e protagonista di importanti campagne militari.
- **Cleopatra**: ultima regina d'Egitto, abile stratega politica e diplomatica, simbolo di potere, cultura e fascino nel mondo antico.
- **Leonardo da Vinci**: genio universale del Rinascimento, artista, inventore e scienziato, noto per la sua inesauribile curiosità e il suo contributo a numerosi campi del sapere.
- **Napoleone Bonaparte**: condottiero e imperatore francese, protagonista della storia europea tra XVIII e XIX secolo, noto per le sue riforme politiche e le campagne militari.
- **Pitagora**: filosofo e matematico dell'antica Grecia, fondatore di una scuola di pensiero che univa scienza, etica e spiritualità.

- **Marie Curie**: fisica e chimica, pioniera nello studio della radioattività, prima donna a vincere un Premio Nobel e unica ad averne ricevuti due in discipline diverse.

- **Albert Einstein**: fisico teorico, autore della teoria della relatività, figura centrale della scienza moderna e simbolo del pensiero scientifico del XX secolo.

3.1 Il Dataset: Progettazione e Generazione della Conoscenza

Uno degli aspetti più critici e distintivi di questo progetto è stata la creazione del dataset. Non esistendo una base di dati strutturata che permettesse un'interazione in prima persona con personaggi storici, si è reso necessario costruire l'intera "conoscenza" del bot da zero.

3.1.1 Generazione tramite Large Language Models (LLM)

Data la complessità nel reperire e formattare grandi quantità di aneddoti, citazioni e dettagli biografici in uno stile narrativo coerente, si è deciso di utilizzare tecniche di **Generazione Sintetica dei Dati** tramite modelli di linguaggio avanzati (LLM).

Il processo ha seguito diverse fasi:

- **Definizione delle Persona**: Per ogni personaggio (Cesare, Cleopatra, Einstein, ecc.), è stato definito un "System Prompt" specifico che istruiva l'LLM a generare contenuti mantenendo un tono e un vocabolario adeguati all'epoca e alla personalità del soggetto (es. stile imperioso per Napoleone, mistico per Pitagora).

- **Generazione della Knowledge Base**: I contenuti storici sono stati estratti e formattati direttamente in strutture dati Python (dizionari), garantendo che ogni risposta fosse scritta in prima persona (*"Io ho scoperto..."*, *"Nella mia epoca..."*).

- **Verifica e Raffinamento**: Sebbene la generazione sia stata automatizzata, ogni blocco di testo è stato revisionato per eliminare eventuali "allucinazioni". Il risultato è un dataset generato dall'IA, ma la struttura e la correttezza sono garantite da un controllo umano rigoroso.

3.1.2 Struttura della Knowledge Base

Il database informativo del chatbot è implementato nel file `knowledge_base.py`. La scelta architetturale è stata quella di utilizzare un dizionario nidificato (`CHARACTERS_DB`) che associa a ogni personaggio una serie di aree tematiche speculari agli intenti del chatbot.

Ogni voce del database comprende:

1. **Metadata Personali**: Nome completo, titoli, anni di vita, stile comunicativo e sesso. Quest'ultimo è fondamentale, permettendo al bot di declinare correttamente i messaggi in base al genere dell'interlocutore storico.

2. **Overview Tematiche**: Per ogni macro-area (*Vita, Studio, Scoperte, Curiosità, Relazioni, Eredità* e *Visione del Futuro*), esiste una voce *overview*. Si tratta di una sintesi narrativa che funge da risposta standard quando l'utente pone domande generiche.

3. **Topic Specifici**: All'interno di ogni categoria, sono stati inseriti dei "sotto-argomenti" (es. *"Rubicone"* sotto *"Vita"* per Cesare) che permettono al bot di suggerire proattivamente all'utente nuovi spunti di conversazione.

4. **Fallback Personalizzati**: Per aumentare l'immersività, sono state generate risposte di "mancata conoscenza" specifiche per ogni personaggio (es. Marie Curie che risponde: *"Il mio laboratorio non ha ancora prodotto dati su questo..."*).

3.1.3 Vantaggi della Generazione Sintetica

L'utilizzo dell'LLM per la creazione del dataset ha permesso di:

- **Uniformità Stilistica**: Garantire che tutte le risposte di un personaggio avessero la stessa "voce".

- **Interattività Dinamica**: Creare collegamenti logici tra gli argomenti (grazie ai suggerimenti dinamici presenti nel DB) che guidano l'utente in un percorso narrativo anziché in una semplice sessione di domande e risposte.

- **Scalabilità**: La struttura è progettata per essere facilmente estesa con nuovi personaggi o nuovi approfondimenti semplicemente aggiungendo chiavi al dizionario Python.

3.2 Files principali

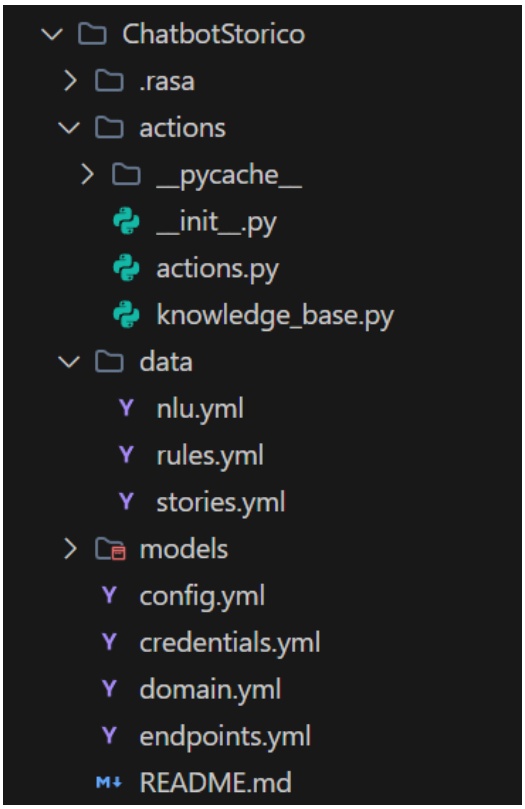


Figura 3: Struttura del progetto

I file costitutivi del progetto sono suddivisi per funzionalità:

- **config.yml**:file che definisce la configurazione del pipeline di elaborazione del linguaggio naturale (NLU) e del dialogo.
- **domain.yml**: Definisce l'"universo" del chatbot, elencando tutti gli intent, le entità, le risposte predefinite (*utterances*) e le azioni custom.
- **credentials.yml**: Gestisce le credenziali per i canali di input/output, nel nostro caso specifico per l'integrazione con **Telegram**.
- **endpoints.yml**: Specifica l'URL del server delle azioni (Action Server), necessario per eseguire il codice Python personalizzato.
- **data/nlu.yml**: Raccoglie i dati di addestramento per la comprensione del linguaggio naturale, associando frasi di esempio ai rispettivi intent.

- **data/rules.yml**: Le Rules definiscono comportamenti rigidi che devono essere seguiti sempre, indipendentemente dal contesto precedente.
- **data/stories.yml**: Le stories sono esempi di dialoghi che insegnano al modello come gestire flussi conversazionali che possono variare.
- **actions/actions.py**: Contiene il codice Python per le *Custom Actions*, inclusa la logica di selezione del personaggio e il recupero delle informazioni storiche.
- **actions/knowledge_base.py**: Un database strutturato (dizionario Python) che funge da base di conoscenza, contenente tutte le informazioni, citazioni e dettagli sui personaggi storici.

3.3 Sviluppo del ChatBot

3.3.1 Inizio e fine chat

La conversazione inizia con l'intent `greet`, che attiva la presentazione dei personaggi disponibili (`action_show_characters`). La chiusura avviene tramite l'intent `goodbye`, che saluta l'utente in modo cordiale.

```
4   - intent: greet
5     examples: |
6       - ciao
7       - salve
8       - buongiorno
9       - buonasera
10      - hey
11      - hello
12      - saluti
13      - come va
14      - ehi
15      - ci sei?
```

Figura 4: Intent di inizio chat

```
17  - intent: goodbye
18    examples: |
19      - arrivederci
20      - addio
21      - a presto
22      - ci vediamo
23      - bye
24      - alla prossima
25      - grazie e ciao
26      - devo andare
27      - chiudo
28      - buonanotte
29      - vado via
30      - fine
31      - basta così
```

Figura 5: Intent fine chat

Successivamente sono riportati esempi di utilizzo per l'inizio della conversazione e due esempi di chiusura della conversazione.

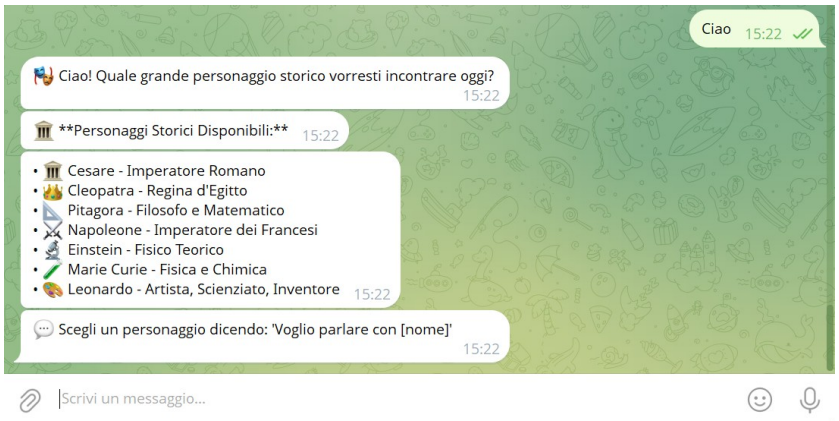


Figura 6: Inizio conversazione

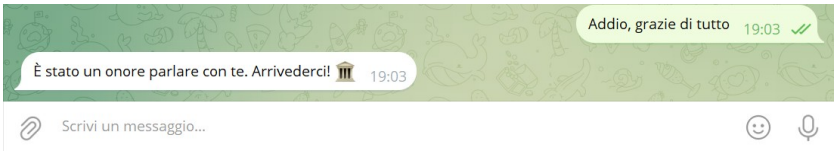


Figura 7: Chiusura conversazione 1



Figura 8: Chiusura conversazione 2

3.3.2 Intent richiesta abilità chatbot

Tramite l'intent `help`, il bot fornisce una guida rapida su come interagire, suggerendo di scegliere un personaggio e indicando i temi disponibili (Vita, Scoperte, Curiosità, ecc.).

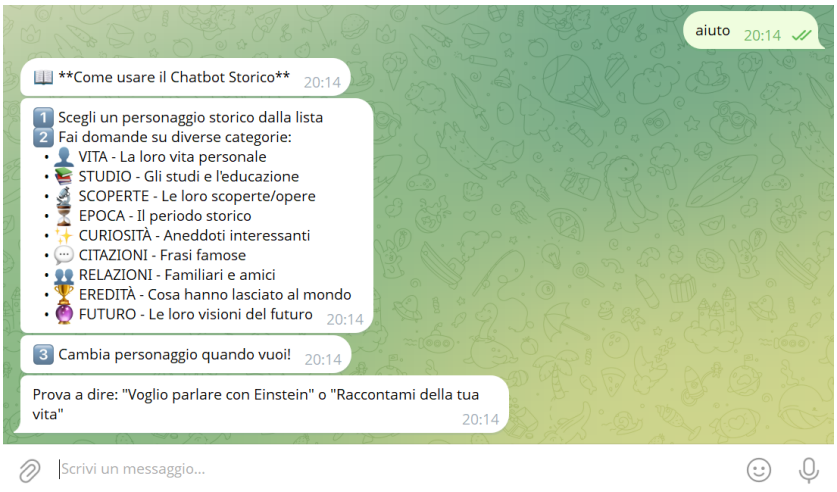


Figura 9: Richiesta aiuto

Per l'implementazione di questa funzionalità è stato sviluppato il seguente intent:

```
749   - intent: help
750     examples: |
751       - aiuto
752       - cosa posso fare
753       - come funziona
754       - comandi
755       - help
756       - cosa posso chiedere
757       - come si usa
758       - istruzioni
759       - guida
760       - cosa puoi fare
761       - dimmi come funziona
762       - hai una guida
763       - spiegami
```

Figura 10: Intent help

3.3.3 Intent di richiesta non valida

Se l'utente inserisce un messaggio che non corrisponde a nessun intent conosciuto o se la confidenza dell'NLU è troppo bassa, entra in gioco il meccanismo di fallback. Nel contesto della selezione del personaggio, se il nome digitato non viene riconosciuto, il bot invoca una logica di riparazione che chiede all'utente di selezionare nuovamente da una lista esplicita tramite la funzione `send_character_selection_menu`.

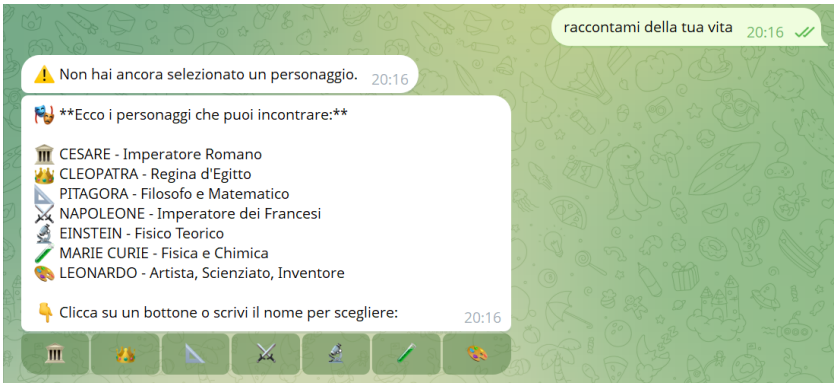


Figura 11: Richiesta non valida

3.3.4 Intent di Fallback e Topic Specifici

Una delle sfide principali in un chatbot storico è gestire domande su argomenti di nicchia non coperti esplicitamente dal flusso principale. L'intent `ask_specific_topic` e la relativa azione `action_risposta_topic_specifico` gestiscono proprio questo scenario. Il sistema analizza il testo dell'utente alla ricerca di parole chiave presenti nella *knowledge base* del personaggio attivo. La logica Python esegue una normalizzazione del testo (rimozione accenti e minuscolo) sia sull'input utente che sulle chiavi del database per garantire un matching robusto. Se trova una corrispondenza (es. "Rubicone" per Cesare o "Radio" per Marie Curie), restituisce il dettaglio specifico contenuto nel dizionario `topics` della categoria corrispondente.

Se invece non viene trovato alcun match valido, il sistema attiva un **fallback personalizzato**: invece di una risposta generica di errore ("Non ho capito"), il bot risponde con una frase coerente con lo stile del personaggio (es. "Le mie pergamene tacciono su questo...") e, tramite la funzione `get_random_suggestions`, suggerisce proattivamente altri 2-3 argomenti casuali di cui può parlare, mantenendo viva la conversazione e guidando l'utente.

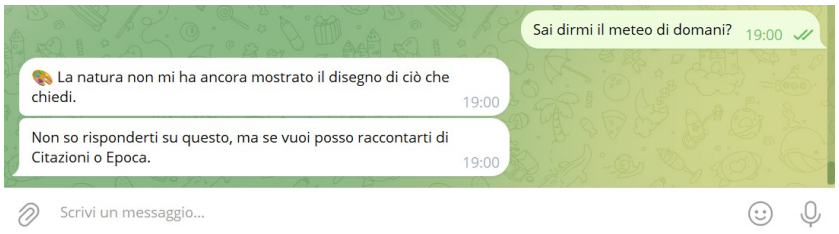


Figura 12: Esempio di fallback personalizzato 1

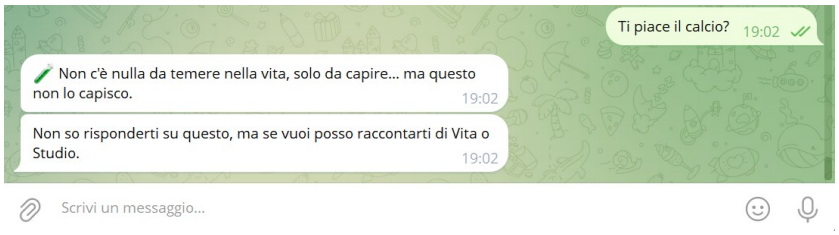


Figura 13: Esempio di fallback personalizzato 2

3.3.5 Intent: select_character

Questo intent è il punto di ingresso per la conversazione. L'utente esprime la volontà di parlare con un personaggio specifico (es. "Voglio parlare con

Cesare"). L'azione associata, `action_select_character`, non si limita a impostare lo slot `selected_character`, ma esegue una ricerca intelligente: prima prova il match diretto tramite entità, poi scansiona manualmente il messaggio alla ricerca dei nomi dei personaggi presenti nel `CHARACTERS_DB`. Una volta validato, restituisce un messaggio di benvenuto personalizzato in base al genere del personaggio (gestito tramite il campo `sesso` nel DB) che include titolo e anni di vita.

3.3.6 Intent Tematici

Per ciascuna delle macro-aree tematiche (Vita, Studio, Scoperte, Epoca, Curiosità, Citazioni, Relazioni, Eredità, Futuro), è stato definito un intent specifico (es. `ask_vita`, `ask_studio`). La gestione di questi intent è centralizzata nella funzione `gestisci_risposta_interattiva` richiamata dalle varie azioni:

1. **Recupero Overview**: Il bot estrae la voce `_overview` corrispondente alla categoria dal database. Questa descrizione è scritta in prima persona per massimizzare l'immedesimazione.

2. **Generazione Suggerimenti**: L'azione scansiona le chiavi disponibili nel dizionario `_topics` correlato. Se presenti, genera una stringa discorsiva (es. "Sarei lieto di narrarti dettagli su: [Topic1] o [Topic2]") per invitare l'utente ad approfondire.

3. **Fallback Categoria**: Se una categoria specifica non ha dati strutturati per quel personaggio, il sistema propone argomenti alternativi per non interrompere il flusso.

Successivamente è riportato l'elenco completo degli intent tematici implementati.

3.3.7 Intent: ask_vita

Permette di ottenere una panoramica biografica del personaggio selezionato, focalizzandone i momenti salienti.

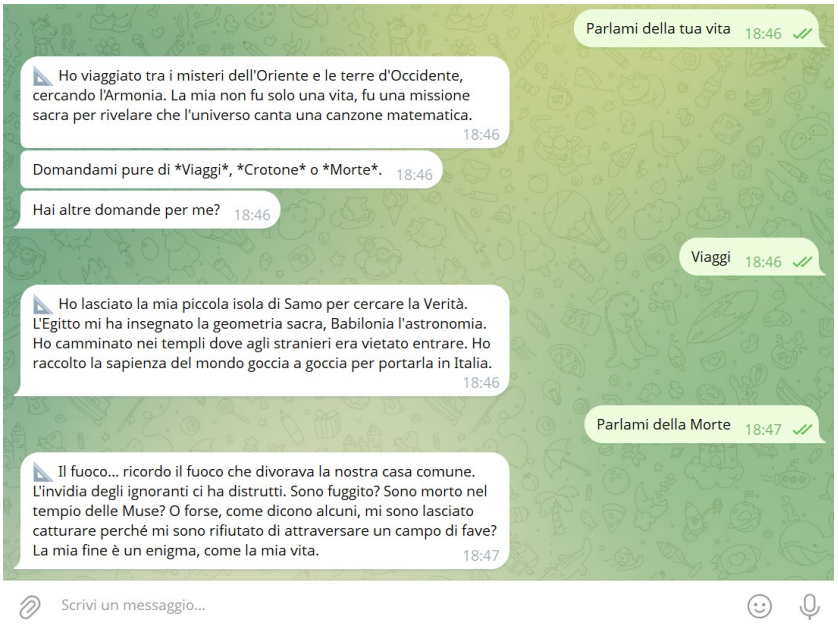


Figura 14: Conversazione sulla vita

Per l'implementazione di questa funzionalità è stato sviluppato il seguente intent:

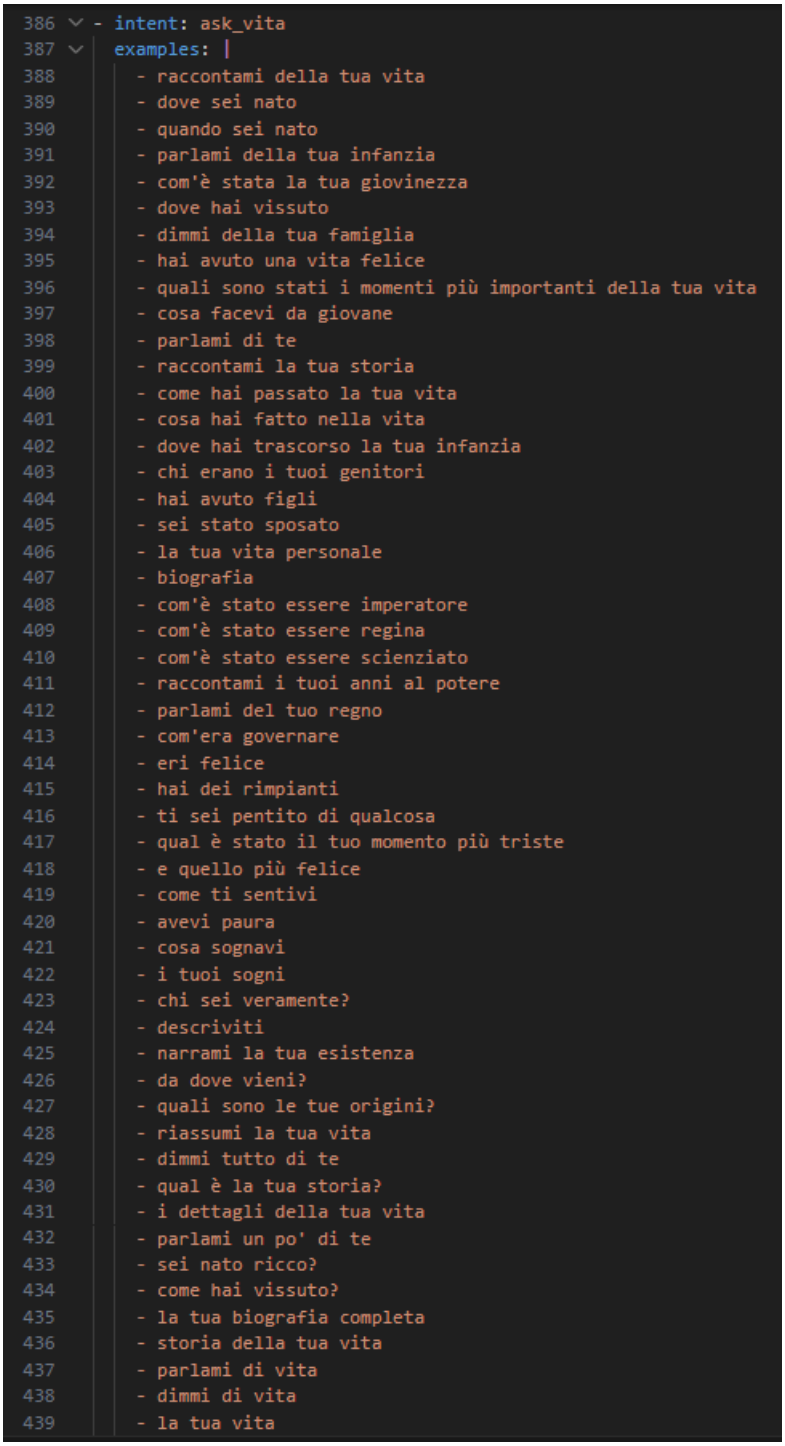


Figura 15: Intent Vita

3.3.8 Intent: ask_studio

Fornisce dettagli sul percorso formativo, l'educazione e le influenze intellettuali che hanno plasmato il personaggio.

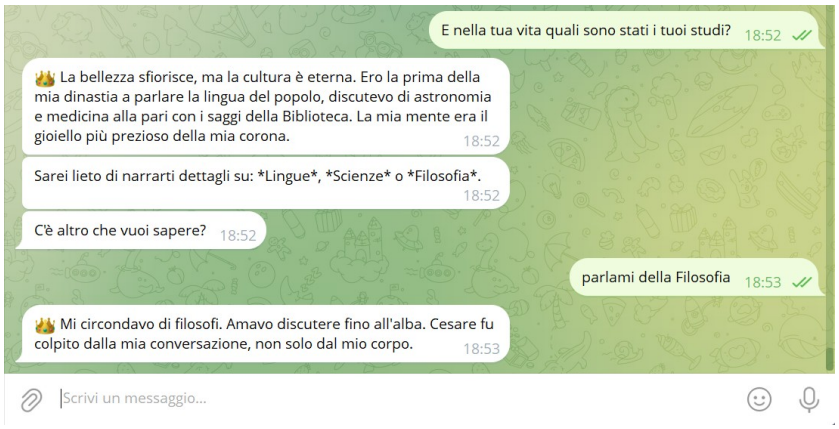


Figura 16: Conversazione sugli studi

Per l'implementazione di questa funzionalità è stato sviluppato il seguente intent:

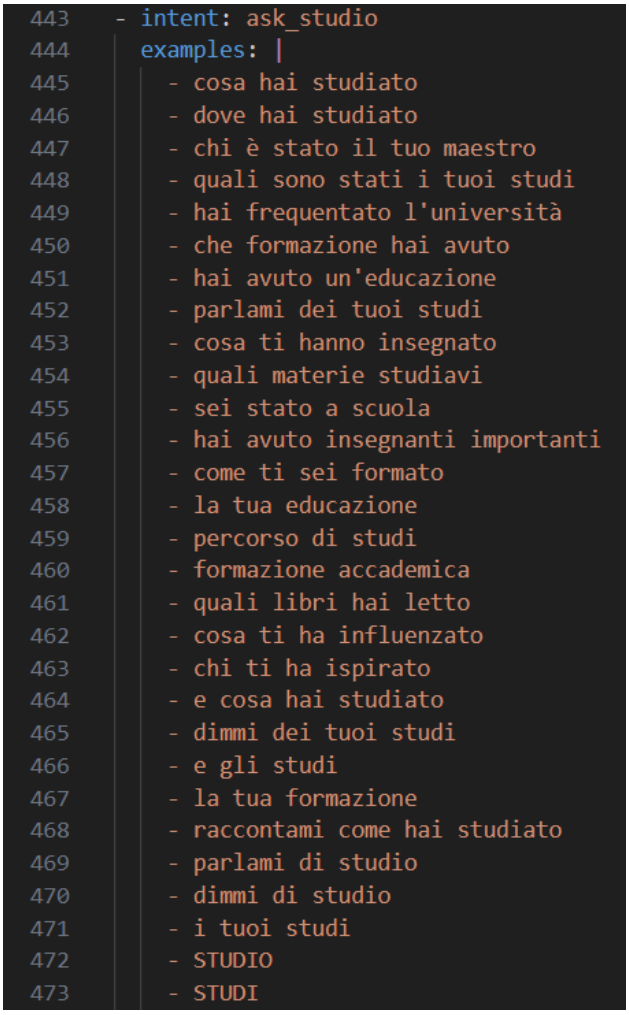


Figura 17: Intent Studio

3.3.9 Intent: ask_scoperte

Dedicato alle scoperte scientifiche, alle invenzioni o alle opere d'arte principali create dalla figura storica.

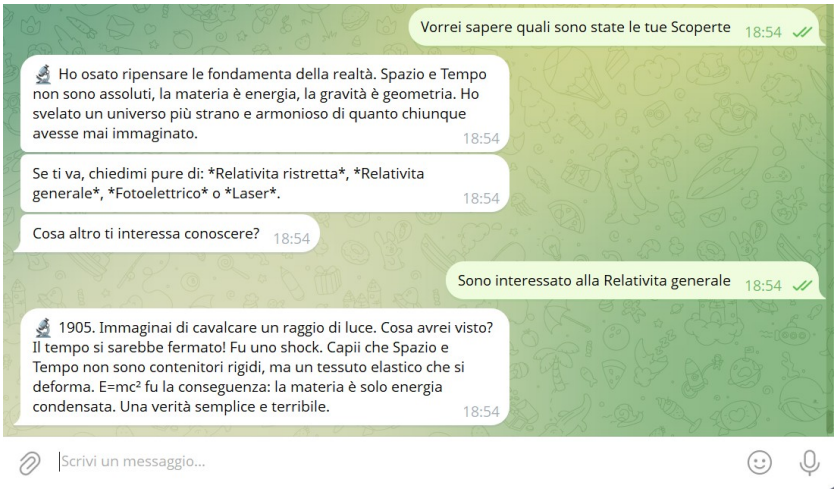


Figura 18: Conversazione sulle scoperte

Per l'implementazione di questa funzionalità è stato sviluppato il seguente intent:

```
475   - intent: ask_scoperte
476     examples: |
477       - quali sono state le tue scoperte
478       - cosa hai scoperto
479       - cosa hai inventato
480       - parlami delle tue invenzioni
481       - quali sono le tue opere principali
482       - cosa hai creato
483       - per cosa sei famoso
484       - quali sono i tuoi lavori
485       - cosa hai realizzato
486       - le tue scoperte scientifiche
487       - i tuoi contributi
488       - cosa hai fatto di importante
489       - quali sono le tue teorie
490       - parlami dei tuoi lavori
491       - le tue opere d'arte
492       - i tuoi scritti
493       - le tue ricerche
494       - cosa hai dato al mondo
495       - i tuoi successi
496       - qual è il tuo capolavoro?
497       - cosa hai lasciato all'umanità?
498       - perché sei sui libri di storia?
499       - la tua opera più grande
500       - cosa hai costruito?
501       - invenzioni
502       - scoperte
503       - teorie
504       - cosa hai capito del mondo?
505       - il tuo contributo alla scienza
506       - il tuo contributo all'arte
507       - per quale motivo sei celebre?
508       - elencami le tue vittorie
509       - parlami delle tue conquiste
510       - cosa hai fatto di speciale?
511       - il tuo lavoro migliore
512       - parlami di scoperte
513       - dimmi di scoperte
514       - cosa hai scoperto?
515       - parlami delle tue scoperte
516       - dimmi qualcosa sulle tue scoperte
517       - SCOPERTE
518       - INVENZIONI
```


Figura 19: Intent Scoperte

3.3.10 Intent: ask_epoca

Risponde a domande sul contesto storico, politico e sociale in cui il personaggio è vissuto, offrendo uno spaccato del tempo.



Figura 20: Conversazione sull'epoca

Per l'implementazione di questa funzionalità è stato sviluppato il seguente intent:

```
521 - intent: ask_epoca
522   examples: |
523     - in che epoca sei vissuto
524     - com'era la tua epoca
525     - che periodo storico hai vissuto
526     - parlami del tuo tempo
527     - cosa succedeva ai tuoi tempi
528     - com'era il mondo quando vivevi
529     - eventi storici della tua epoca
530     - cosa accadeva nel tuo periodo
531     - il contesto storico
532     - la tua era
533     - quali eventi hai vissuto
534     - com'era la società
535     - come viveva la gente
536     - quali erano i costumi
537     - tecnologia del tempo
538     - cultura dell'epoca
539     - politica del periodo
540     - guerre del tuo tempo
541     - chi governava
542     - parlami di epoca
543     - dimmi di epoca
544     - com'era la vita ai tuoi tempi?
545     - nel tuo periodo
546     - la tua epoca
547     - EPOCA
548     - AI TUOI TEMPI
```

Figura 21: Intent Epoca

3.3.11 Intent: ask_curiosita

Attiva la narrazione di aneddoti, vizi, virtù o fatti insoliti che rendono il personaggio più "umano" e vicino all'utente.

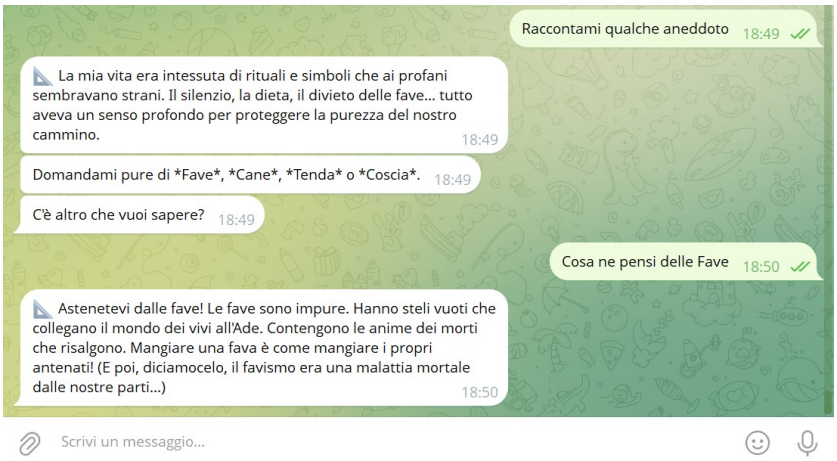


Figura 22: Conversazione sulle curiosità

Per l'implementazione di questa funzionalità è stato sviluppato il seguente intent:

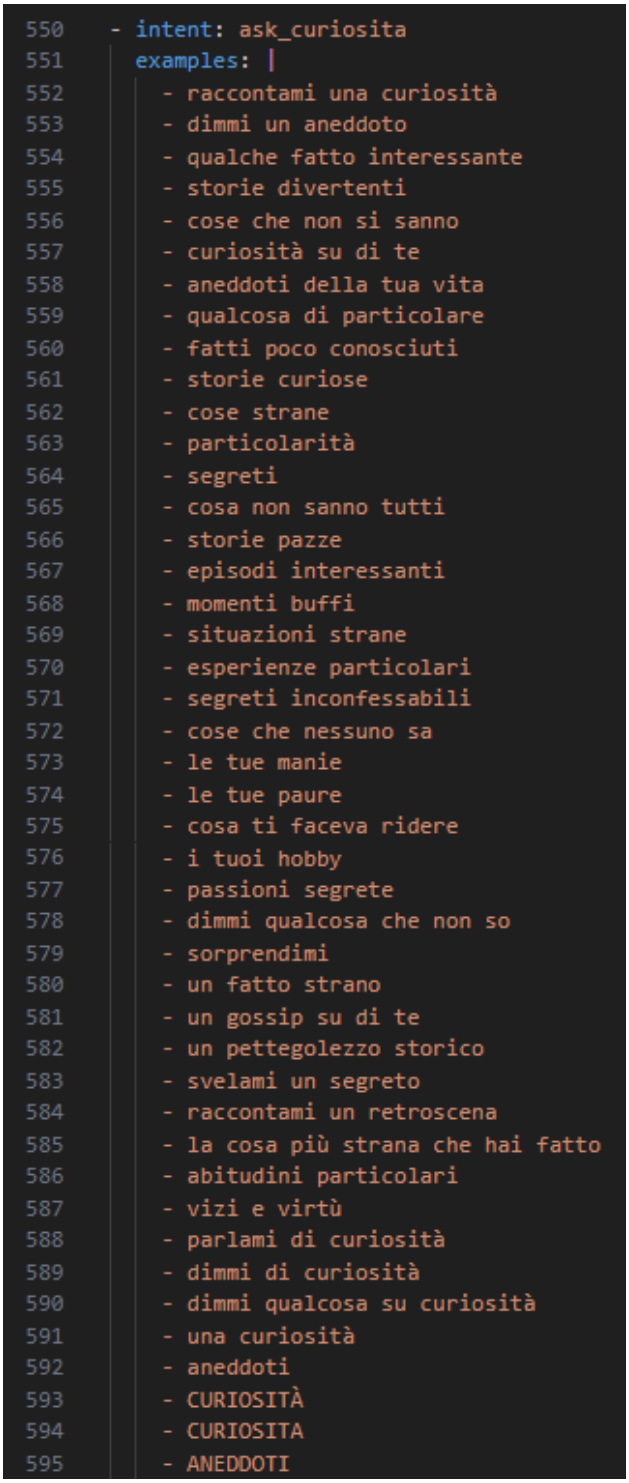


Figura 23: Intent Curiosità

3.3.12 Intent: ask_citazioni

Il bot risponde citando frasi celebri, aforismi o passaggi significativi tratti da opere o discorsi del personaggio.

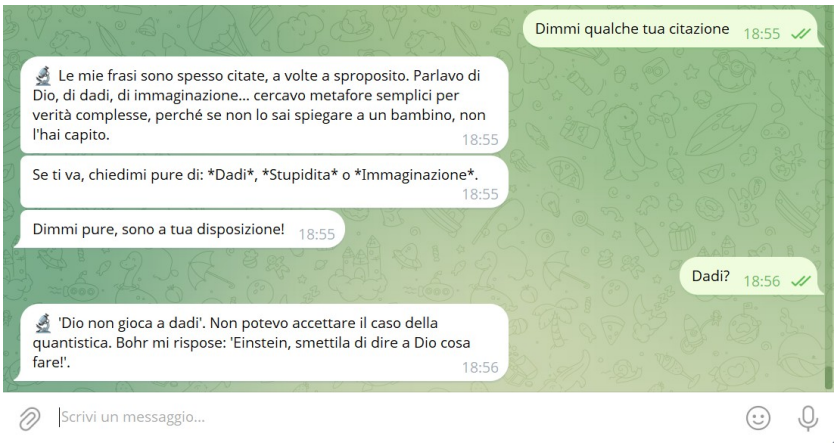


Figura 24: Conversazione sulle citazioni

Per l'implementazione di questa funzionalità è stato sviluppato il seguente intent:

```
598  - intent: ask_citazioni
599    examples: |
600      - qual è la tua frase più famosa
601      - dimmi una citazione
602      - cosa dicevi spesso
603      - frasi celebri
604      - le tue parole famose
605      - cosa hai detto di importante
606      - citazioni famose
607      - aforismi
608      - massime
609      - parole memorabili
610      - frasi che hai detto
611      - i tuoi detti
612      - cosa ripetevi sempre
613      - le tue frasi preferite
614      - pensieri famosi
615      - discorsi importanti
616      - parole sagge
617      - le tue parole
618      - dimmi qualche tua citazione
619      - e le citazioni
620      - qualche frase famosa
621      - parlami di citazioni
622      - dimmi di citazioni
623      - dimmi qualche tua frase celebre
624      - una tua citazione
625      - CITAZIONI
626      - FRASI CELEBRI
```

Figura 25: Intent Citazioni

3.3.13 Intent: ask_relazioni

Esplora la vita privata, i legami familiari, le amicizie e le rivalità che hanno segnato l'esistenza della figura storica.

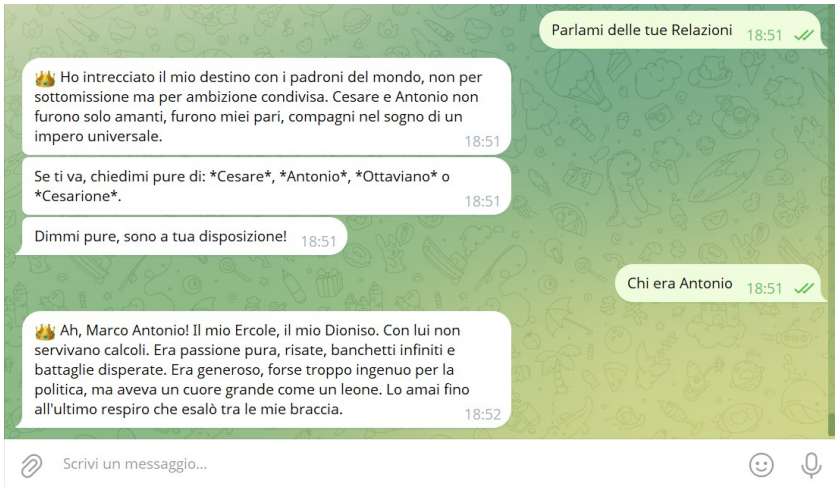


Figura 26: Conversazione sulle relazioni

Per l'implementazione di questa funzionalità è stato sviluppato il seguente intent:

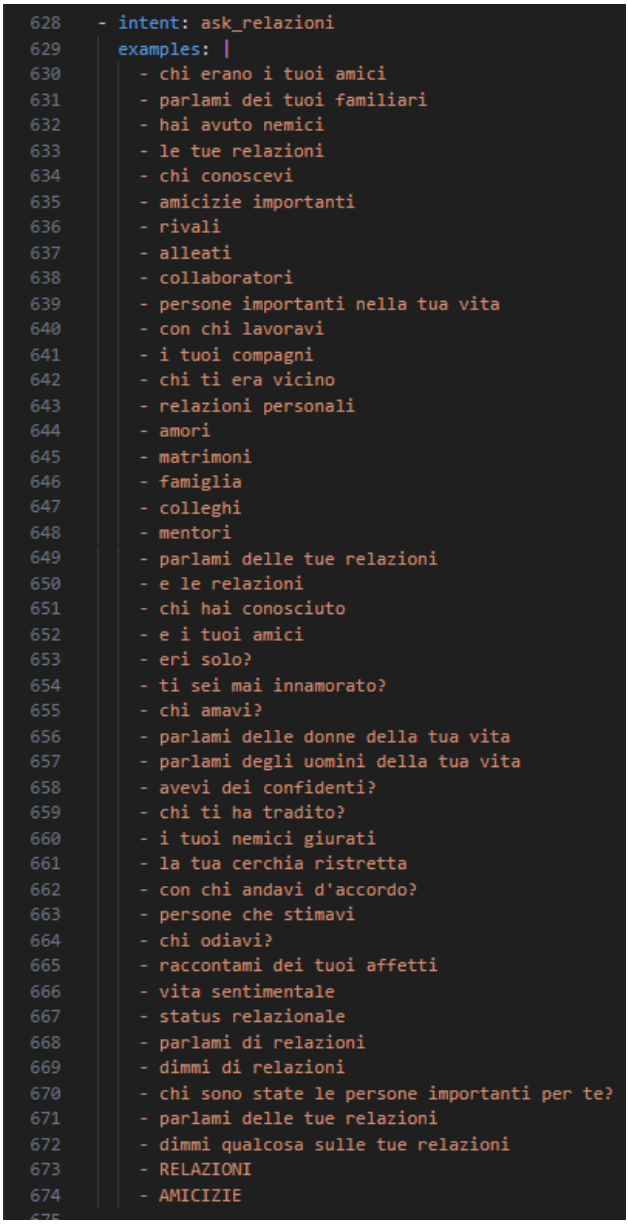


Figura 27: Intent Relazioni

3.3.14 Intent: ask_eredita

Descrive l'impatto a lungo termine del personaggio e come viene ricordato e celebrato nel mondo contemporaneo.

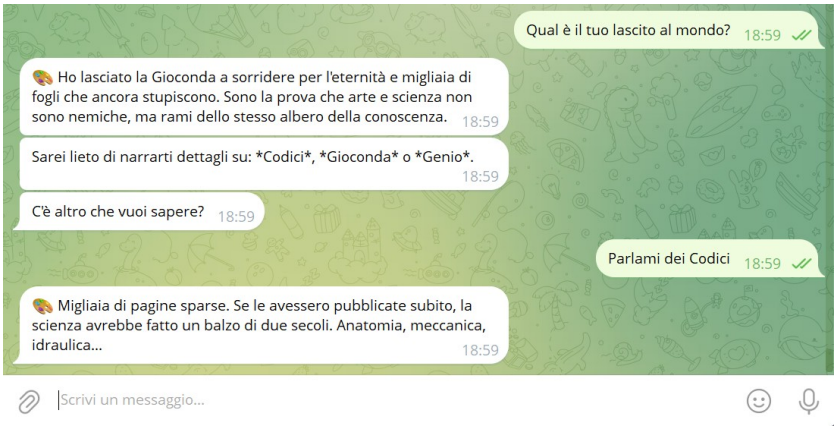


Figura 28: Conversazione sull'eredità

Per l'implementazione di questa funzionalità è stato sviluppato il seguente intent:

```
- intent: ask_eredita
  examples: |
    - qual è la tua eredità
    - cosa hai lasciato
    - il tuo lascito
    - cosa rimane di te
    - per cosa sei ricordato
    - il tuo contributo alla storia
    - cosa hai dato all'umanità
    - la tua influenza
    - cosa è rimasto delle tue opere
    - eredità storica
    - impatto sul mondo
    - cosa è cambiato grazie a te
    - come hai influenzato il futuro
    - il tuo lascito culturale
    - cosa sopravvive di te
    - memoria storica
    - come sei ricordato oggi
    - parlami di eredità
    - dimmi di eredità
    - cosa hai lasciato al mondo?
    - EREDITÀ
    - EREDITA
    - LASCITO
```

Figura 29: Intent Eredità

3.3.15 Intent: ask_futuro

Sfrutta i dati storici per ipotizzare o riportare le visioni e le speranze che il personaggio aveva per i secoli a venire.

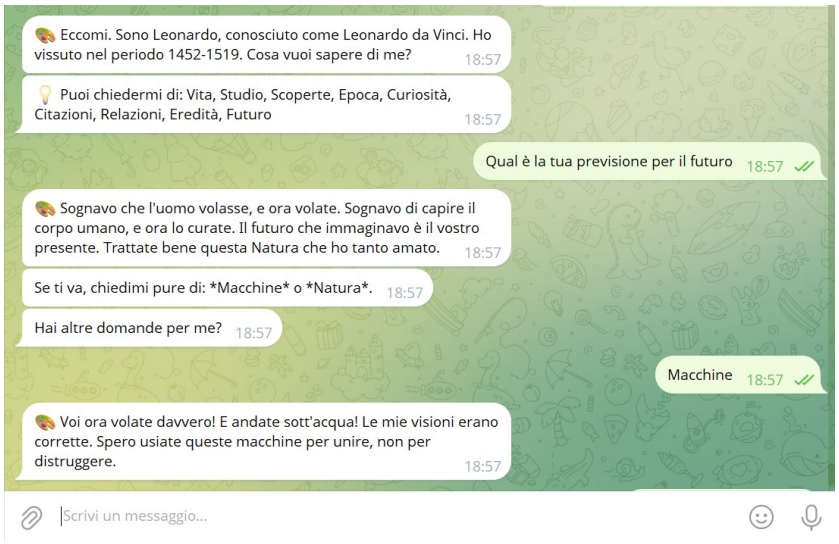


Figura 30: Conversazione sul futuro

Per l'implementazione di questa funzioalità è stato sviluppato il seguente intent:

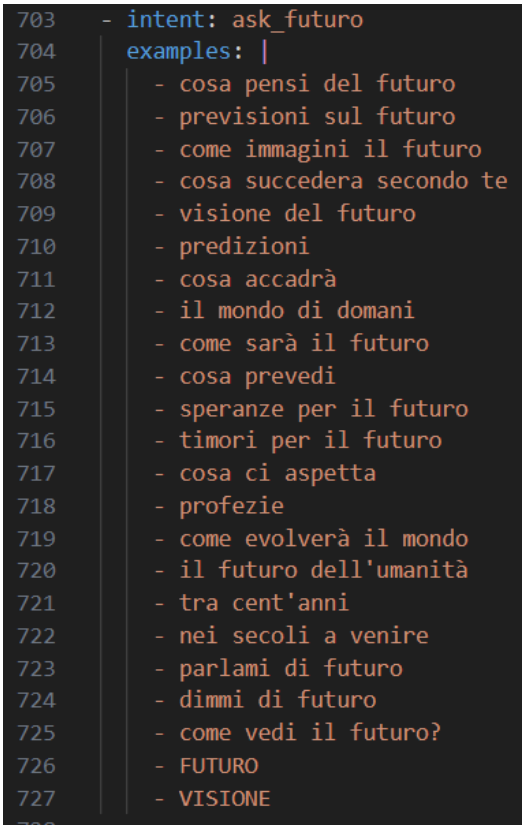


Figura 31: Intent Futuro

3.3.16 Intent: change_character

Permette all'utente di resettare la sessione corrente per scegliere un nuovo interlocutore storico. L'azione `action_reset_character` azzera gli slot `selected_character` e `character_style`, invocando poi la lista dei personaggi. Questo garantisce che la "personalità" del bot venga resettata correttamente prima della nuova scelta.

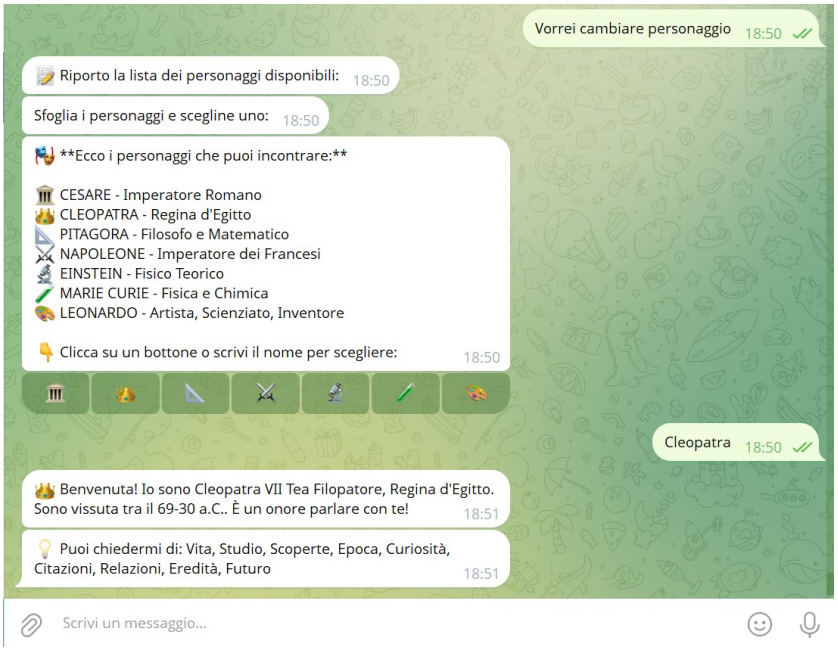


Figura 32: Cambio del personaggio con lista

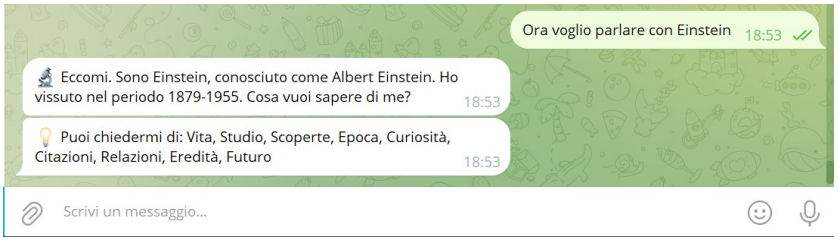


Figura 33: Cambio del personaggio diretto

Per l'implemetnazione di questa funzionalità è stato sviluppato il seguente intent:

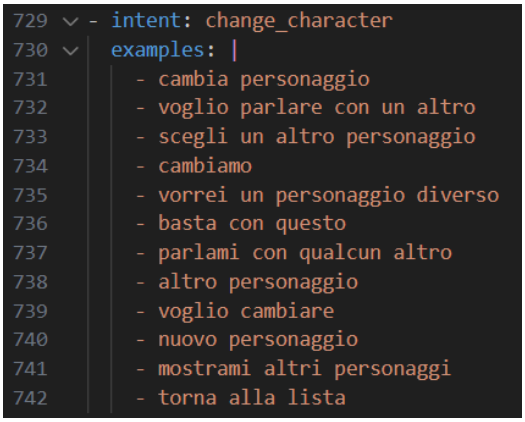


Figura 34: Intent Cambio Personaggio

3.4 Rules e Stories

Per gestire il flusso del dialogo, Rasa utilizza due concetti fondamentali: **Rules** (Regole) e **Stories** (Storie), entrambi definiti nei file YAML all'interno della cartella `data/`.

3.4.1 Rules

Le **Rules** (`data/rules.yml`) descrivono porzioni di conversazione che devono seguire sempre la stessa logica rigida, indipendentemente dal contesto precedente. Sono utilizzate per gestire comportamenti "stateless" o funzionali che non devono essere influenzati dalla storia del dialogo. Nel progetto ChronoBot, le regole sono state utilizzate per:

- **Saluti**: Rispondere sempre a un "ciao" con un saluto e a un "addio" con un messaggio di chiusura.

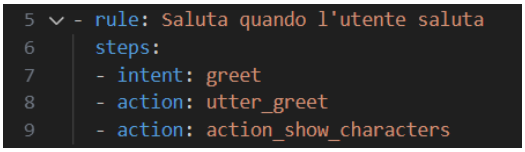


Figura 35: Inizio Conversazione

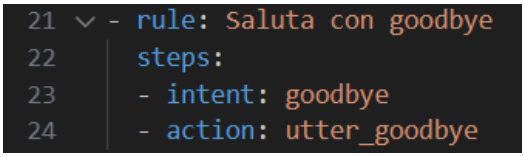


Figura 36: Chiusura conversazione

- **Gestione Aiuto**: Fornire le istruzioni d'uso ogni volta che viene invocato l'intent `help`.

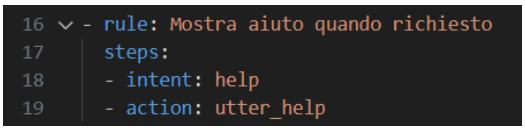


Figura 37: Richiesta aiuto

- **Loop Domanda-Risposta**: Una regola fondamentale (*Rispondi a domanda su [categoria]*) assicura che dopo ogni risposta tematica (es. Vita), il bot chieda sempre "C'è altro che vuoi sapere?" (`utter_ask_another_question`), mantenendo attiva l'interazione.

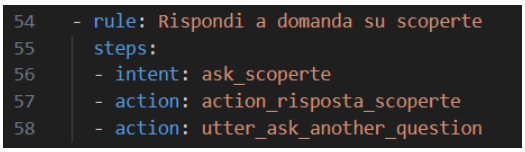


Figura 38: Esempio Domanda-Risposta

3.4.2 Stories

Le **Stories** (`data/stories.yml`) sono invece esempi di conversazioni reali che servono ad addestrare il modello di Machine Learning a gestire i flussi di dialogo più complessi e variati. A differenza delle regole, le storie insegnano al bot a generalizzare e a gestire il contesto.

- **Conversazione Completa**: Un flusso tipico che va dal saluto iniziale, alla scelta del personaggio, fino a una serie di domande su vari argomenti e infine ai saluti.

```yaml
- story: conversazione completa con personaggio
  steps:
  - intent: greet
  - action: utter_greet
  - action: action_show_characters
  - intent: select_character
  - action: action_select_character
```

Figura 39: Conversazione completa

- **Cambio Personaggio**: Una storia che insegna al bot come gestire un utente che, nel mezzo di una conversazione, decide di passare a un altro interlocutore storico, garantendo che il passaggio avvenga in modo fluido.

```yaml
- story: cambio personaggio durante conversazione
  steps:
  - intent: select_character
  - action: action_select_character
  - intent: change_character
  - action: action_reset_character
  - action: action_show_characters
  - intent: select_character
  - action: action_select_character
```

Figura 40: Cambio Personaggio

4 Esecuzione del chatBot

4.1 Esecuzione nell'ambiente virtuale

ChronoBot opera in un ambiente virtuale Python (es. Conda). Il ciclo di esecuzione prevede:

```
# Addestramento del modello
rasa train

# Avvio del server delle azioni (necessario per la logica
    custom)
rasa run actions

# Avvio della chat in locale
rasa shell
```

4.2 Connessione a Telegram

L'integrazione con Telegram avviene tramite il file `credentials.yml`.
Per farlo bisogna:

1. Attivare un account Telegram

2. Iniziare la chat con *BotFather* lanciando il comando "". Alla fine di tale procedura 'e importante inserire il token fornito da BotFather all'interno del file credentials.yml,

3. Attivare un'account su *Ngrok* e scaricare l'applicativo, registrarsi e inserire1 auth token

4. Avviare il tunneling:

```
    ngrok http 5005

```

5. Copiare il dominio fornito da *Ngrok* in `credentials.yml` all'interno del campo webhook_url

6. Eseguire su due terminali diversi:

```
    #Terminale 1
    rasa run action

    #Terminale 2
    rasa run --enable-api --cors "*" --debug

    #Se ci sono errori con ngork eseguire
    pip uninstall aiogram -y
    pip install aiogram==2.25.1

```

7. Cercare il bot @UsernameBot su Telegram e iniziare la conversazione

```
18  telegram:
19    access_token: "YOUR_BOT_TOKEN_HERE"
20    verify: "YOUR_BOT_NAME"
21    webhook_url: "YOUR_WEBHOOK_URL"
22
23
24  rasa:
25    server: "http://localhost:5005"
```

Figura 41: Esempio di file credentials.yml

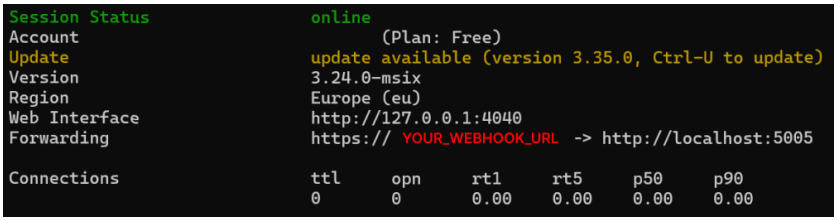


Figura 42: Connesione Ngrok

Elenco delle figure

1 Logo Rasa . . . 3
2 Logo ChronoBot . . . 5
3 Struttura del progetto . . . 8
4 Intent di inizio chat . . . 9
5 Intent fine chat . . . 10
6 Inizio conversazione . . . 10
7 Chiusura conversazione 1 . . . 11
8 Chiusura conversazione 2 . . . 11
9 Richiesta aiuto . . . 11
10 Intent help . . . 12
11 Richiesta non valida . . . 12
12 Esempio di fallback personalizzato 1 . . . 13
13 Esempio di fallback personalizzato 2 . . . 13
14 Conversazione sulla vita . . . 15
15 Intent Vita . . . 16
16 Conversazione sugli studi . . . 17
17 Intent Studio . . . 18
18 Conversazione sulle scoperte . . . 19
19 Intent Scoperte . . . 20
20 Conversazione sull'epoca . . . 21
21 Intent Epoca . . . 21
22 Conversazione sulle curiosità . . . 22
23 Intent Curiosità . . . 23
24 Conversazione sulle citazioni . . . 24
25 Intent Citazioni . . . 25
26 Conversazione sulle relazioni . . . 26
27 Intent Relazioni . . . 27
28 Conversazione sull'eredità . . . 28
29 Intent Eredità . . . 28
30 Conversazione sul futuro . . . 29
31 Intent Futuro . . . 30
32 Cambio del personaggio con lista . . . 31
33 Cambio del personaggio diretto . . . 31
34 Intent Cambio Personaggio . . . 32
35 Inizio Conversazione . . . 32
36 Chiusura conversazione . . . 33
37 Richiesta aiuto . . . 33
38 Esempio Domanda-Risposta . . . 33
39 Conversazione completa . . . 34

40 Cambio Personaggio 34
41 Esempio di file credentials.yml 35
42 Connesione Ngrok 36